Project Report —Paradox of Hands

Name: Harshit Singhal (25MIM10195)
Date: 23 November 2025
Project name: Paradox of Hands
Language / Tools: Python 3, `tkinter`, `random`, `threading`, `time`

---

1. Project summary

This small desktop application implements a human vs CPU Rock-Paper-Scissors game with a simple GUI built using `tkinter`. The game runs round-by-round, shows ASCII art for the player's and CPU's choices, keeps score, tracks rounds, and ends when either player or CPU reaches a preset win limit (default 5). The interface aims to be playful and easy for beginners to both play and read the code.

---

2. Objectives

- Create a friendly GUI for Rock-Paper-Scissors.
- Demonstrate class design and separation of game logic from UI.
- Use threading to avoid freezing the UI during simulated CPU “thinking”.
- Maintain a history of rounds for potential extension (replay, undo, export).
- Keep the project simple and educational — good for learning `tkinter` basics.

---

3. High-level design & components

Game logic (class `GameRPS`)

- **State variables**
  - `p`, `c`, `d` — integer counters for player wins, CPU wins, draws.
  - `r` — current round number (starts at 1).
  - `h` — history list storing tuples `(round, player_choice, cpu_choice, result)`.
  - `wl` — win limit (default 5).
- **Resources**
  - `pics` — dictionary containing ASCII-art strings for `"rock"`, `"paper"`, `"scissors"`.
  - `ops` — list of available choices.
- **Methods**
  - `calc(u, cc)` — computes outcome (`"win"`, `"lose"`, `"draw"`) given user `u` and cpu `cc`.
  - `save(u, cm, rr)` — append round data to history `h`.

    - `done()` — returns `"p"` if player reached win limit, `"c"` if CPU reached it, otherwise `None`.

GUI (Tkinter)

- Main window: title `"Paradox of Hands"`, geometry `530x535`.
- Labels:
    - `lb1` — shows player's choice.
    - `lb2` — shows CPU's choice.
    - `lb3` — shows textual result of current round.
    - `lb4` — shows score (Player - CPU).
    - `lb5` — shows round number.
- Text widget `txt` — displays ASCII art side-by-side for player and CPU choices, plus status messages.
- Frame `fr` with three Buttons — `"Rock"`, `"Paper"`, `"Scissors"`; each triggers `pressed(choice)`.
- `pressed(x)` starts a daemon `threading.Thread` to run `goRound(x)` so the UI remains responsive while `time.sleep` simulates CPU thinking.

---

4. Program flow (step-by-step)

1. User clicks one of the three buttons.
2. `pressed(choice)` spawns a daemon thread to run `goRound(choice)`.
3. `goRound`:
    - Updates `lb1` to show the user's choice.
    - Shows staged messages in `txt` while sleeping (`0.27s`, `0.4s`) to simulate thinking.
    - Picks CPU choice using `random.choice(g.ops)`.
    - Updates `lb2` and inserts ASCII arts into `txt`.
    - Calls `g.calc(user, cpu)` to determine the round outcome.
    - Updates counters and `lb3`/`lb4`.
    - Saves the round into `g.h`.
    - Checks `g.done()` to determine if the game ended — if yes, opens a `Toplevel` dialog showing final result and a button that closes the entire app; otherwise increments round `g.r` and updates `lb5`.

---

5. Key code excerpts explained

- `calc` uses a small mapping `xx = {"rock":"scissors","paper":"rock","scissors":"paper"}` to check wins: if the mapped losing choice of the user's pick equals the CPU choice — user wins.
- `goRound` runs in a separate thread to avoid blocking the mainloop while using `time.sleep` for animation-like delays.
- `g.save` records every round for later analysis (possible extension: export to file).

---

6. User experience (UX) behavior

- Visual feedback: labels update for immediate feedback; ASCII art gives a retro charm.
- CPU thinking effect: staged "cpu choosing..." and "cpu hmm..." messages add suspense.
- End-of-game dialog: a small window announces the winner and final score, with an “ok” button that closes the app.

---

7. Running the project

Requirements:

- Python 3 (3.6+ recommended)
- Standard library modules only: `tkinter`, `random`, `time`, `threading`

How to run:

1. Save the script to a file, e.g. `Paradox of Hands_gui.py`.
2. From terminal/command prompt: P`aradox of Hands_gui.py`
3. Click any of the three buttons to play.

---

8. Test cases & expected outcomes

- Click the same choice repeatedly while CPU selection is random — scores should change accordingly; no crashes expected.
- Simulate draw: click `"rock"` while CPU chooses `"rock"` — `lb3` should read `Result: Draw`, and `g.d` should increment.
- Win condition: play until either `g.p` or `g.c` reaches `g.wl` (default 5); a `Toplevel` window should appear announcing the winner and final score, and the "ok" closes the app.

---

9. Known limitations / issues

- **Hard exit:** The final dialog's OK button calls `root.destroy()`, which force-closes the application. If you wanted to restart instead, additional logic is needed.
- **Thread-safety:** The code updates `tkinter` widgets from a background thread. While it often works, updating GUI elements from threads is not recommended and can cause instability on some platforms. Best practice is to use `root.after(...)` to schedule UI updates back on the main thread.

- **No sound/animation:** Only ASCII art and sleep-based text are used. More engaging visuals are possible.
- **Fixed win limit:** `wl` is a class attribute default (5); no UI control to change it.
- **No persistence/export:** The history `h` is kept only in memory.

---

10. Suggested improvements & future work

1. **Thread-safe UI updates:** Replace direct widget updates from worker thread with `root.after(...)` queued calls.
2. **Config screen:** Allow user to set `win limit`, choose round mode (best of N), or toggle CPU difficulty.
3. **Restart/Play again:** Instead of closing the app at the end, present options: Play Again, View History, or Quit.
4. **Graphical icons:** Replace ASCII art with small images (PNG) to modernize the UI.
5. **Sound effects & animations:** Add brief animations or sound on win/lose/draw.
6. **Statistics & analytics:** Export `g.h` to CSV, show win percentage, streaks, etc.
7. **AI CPU:** Implement weighted CPU choices that adapt to player's tendencies (simple Markov or probability tracking).
8. **Mobile-friendly GUI or web version:** Port to a web UI (Flask + JS) or Kivy for cross-platform touch support.

---

11. Code quality & learning notes

- The code is compact, readable, and beginner-friendly.
- Methods are small and focused (good).
- Using a `GameRPS` class separates logic from UI — this improves maintainability.
- Consider adding docstrings and type hints for clarity and static analysis support.
- Add exception handling around UI and game logic to gracefully handle unexpected errors.

---

12. Example extension: safer UI update pattern

Replace thread GUI writes with `root.after(...)`. Example pattern for updating `lb2`:

```
# inside goRound (worker thread)
cm = random.choice(g.ops)
root.after(0, lambda: lb2.config(text="CPU: "+cm.capitalize()))
```

This ensures the actual label updates run on the main thread.

---

13. Conclusion

This project is a great small exercise in GUI programming, threads, and encapsulating game logic. It’s ideal as a teaching project and has clear, easy extensions (graphics, AI, stats). With a few improvements (thread-safe UI updates and restart options), it becomes a polished little game you can show in a portfolio or expand into a more feature-rich mini app.
